

차 례

Blacklist 오프라인 설치 가이드	1
1. 설치 전 요구사항	1
2. 릴리스 공개키 등록	1
3. 최종 패키지 서명과 체크섬 확인	1
4. 최초 설치	1
5. 설치 결과 확인	2
6. WARP 프록시	2
7. 업그레이드	2

Blacklist 오프라인 설치 가이드

1. 설치 전 요구사항

- Linux x86_64 호스트
- root 또는 sudo 권한
- Docker Engine 과 Docker Compose v2
- HTTPS 443 포트 사용 가능
- Docker 데이터 디렉터리에 충분한 여유 공간
- sha256sum, openssl, python3, curl, tar, gpg, pgpv
- 조직의 인증된 별도 채널로 받은 릴리스 공개키와 40 자리 fingerprint

현재 패키지는 Docker 와 Compose 가 이미 설치된 호스트를 기본 전제로 합니다. prereqs/에 Docker tarball 과 Compose 바이너리가 모두 들어 있는 패키지만 베어 호스트 설치를 지원합니다.

2. 릴리스 공개키 등록

일반 설치와 업그레이드는 detached signature 를 필수 검증합니다. 공개키는 패키지와 같은 채널에서 신뢰하지 말고, 조직의 변경관리 시스템이나 별도 인증된 채널로 받아 fingerprint 를 독립적으로 대조합니다.

```
gpg --batch --dearmor --output release-pubkey.gpg release-public-key.asc
gpg --show-keys --with-colons release-public-key.asc | awk -F: '$1 == "fpr" { print $10; exit }'
sudo install -d -o root -g root -m 0755 /etc/blacklist
sudo install -o root -g root -m 0644 release-pubkey.gpg /etc/blacklist/release-pubkey.gpg
rm -f release-pubkey.gpg
```

출력된 fingerprint 가 별도 인증 채널의 값과 정확히 일치하지 않으면 설치하지 않습니다. 패키지 내부 파일이나 릴리스와 함께 전달된 값만으로 fingerprint 를 신뢰해서는 안 됩니다.

3. 최종 패키지 서명과 체크섬 확인

배포 시 받은 tarball, .tar.gz.asc, .tar.gz.sha256 파일을 같은 디렉터리에 둡니다. 압축을 풀거나 패키지 내부 스크립트를 실행하기 전에 호스트 keyring 으로 최종 tarball 서명을 먼저 검증합니다.

```
gpgv --keyring /etc/blacklist/release-pubkey.gpg blacklist-<버전>.tar.gz.asc blacklist-<버전>.tar.gz
sha256sum -c blacklist-<버전>.tar.gz.sha256
tar -xzf blacklist-<버전>.tar.gz
cd blacklist-<버전>
```

서명 또는 체크섬 검증이 실패하면 압축을 풀거나 설치를 실행하지 않습니다.

4. 최초 설치

설치기는 호스트의 /etc/blacklist/release-pubkey.gpg 로 패키지 매니페스트 서명을 검증하고 이미지 체크섬을 다시 확인한 뒤, 대상 호스트에서 관리자 비밀번호와 필수 암호화 키를 생성합니다. 신규 설치와 업그레이드의 기본 외부 TLS mode 는 provided 입니다.

```
sudo bash install.sh --check-secrets
sudo install -d -o 1001 -g 1001 -m 0700 /etc/blacklist/frontend-tls
sudo install -o 1001 -g 1001 -m 0600 server.key /etc/blacklist/frontend-tls/server.key
sudo install -o 1001 -g 1001 -m 0644 server.crt /etc/blacklist/frontend-tls/server.crt
sudoedit /etc/blacklist/.env
sudo bash install.sh
```

.env 에서 실제 접속 FQDN 또는 IP 를 설정합니다. 인증서 SAN 은 이 값과 일치해야 합니다.

```
FRONTEND_TLS_MODE=provided
FRONTEND_TLS_SERVER_NAME=blacklist.example.com
FRONTEND_BIND_ADDRESS=0.0.0.0
```

개발용 자체서명 모드는 명시적으로만 사용할 수 있고 installer 가 loopback bind 를 강제합니다.

```
FRONTEND_TLS_MODE=self-signed
FRONTEND_BIND_ADDRESS=127.0.0.1
```

관리자 비밀번호는 최초 설치 완료 시 한 번만 출력됩니다. 즉시 비밀번호 관리자에 저장하십시오. 원본은 권한이 제한된 /etc/blacklist/.env 에 보관됩니다.

설치 전에 번들 무결성만 읽기 전용으로 검증할 수 있습니다. 대상 .env 가 있으면 runtime security posture 도 함께 검증합니다.

```
sudo bash install.sh --verify-only
```

--verify-only 에서 설치와 동일한 서명 검증을 강제하려면 다음 명령을 사용합니다.

```
sudo bash install.sh --verify-only --require-signature
```

5. 설치 결과 확인

```
sudo docker compose --env-file /etc/blacklist/.env -f docker-compose.yml ps
curl --insecure --fail https://localhost/health
```

정상 상태는 다음과 같습니다.

- blacklist-app, blacklist-collector, blacklist-frontend, blacklist-postgres, blacklist-redis 가 모두 healthy
- 호스트 공개 포트는 frontend 의 443 하나
- PostgreSQL 5432, Redis 6379, Collector 8545, Flask 2542 는 호스트에 미공개
- 미인증 보호 API 는 HTTP 401 반환

6. WARP 프록시

WARP 는 개발 overlay 전용입니다. make dev 는 Collector 에 다음 값을 전달합니다.

```
WARP_ENABLED=true
WARP_PROXY_URL=http://host.docker.internal:40000
```

호스트 WARP proxy 가 Docker 브리지에서 접근 가능해야 합니다. 일반 설치와 release overlay 는 WARP 를 항상 비활성화하며 운영 수집은 직접 연결을 사용합니다.

7. 업그레이드

새 패키지를 별도 디렉터리에 풀고 새 install.sh 를 실행합니다. 기존 /etc/blacklist/.env, /etc/blacklist/release-pubkey.gpg, Docker 볼륨을 유지해야 저장된 자격증명을 계속 복호화하고 릴리스 서명을 검증할 수 있습니다. 기존 env 에 FRONTEND_TLS_MODE 가 없으면 installer 가 provided 로 마이그레이션하므로 실행 전에 server certificate 와 FRONTEND_TLS_SERVER_NAME 을 준비해야 합니다.

```
cd blacklist-<새 버전>
sudo bash install.sh
```

--stop-all-containers 는 호스트의 모든 컨테이너를 중지하므로 일반 설치와 업그레이드에서 사용하지 않습니다.